STUDENT GRADE MANAGEMENT SYSTEM

Individual Programming Project & Software Process Measurement

Course / Project	Software Quality and Improvement
Lecturer	Dr. Tesfaye Biru
Programming language	Elixir
Application type	Command-line application
Student name	Matan Workneh
ID number	0143
Section	2

Project objective

Develop a command-line program that accepts a student's name and a variable number of subject marks, validates the inputs, calculates the total and average, assigns a letter grade, and displays the results. The project also applies process measurement by estimating work before each job.

1. Project Overview

The Student Grade Management System is a small Elixir command-line program. The user enters a student name, chooses how many subjects to record, and enters each subject name and mark. The program validates the entries and displays a summary.

1.1 Scope

Included: one student record per run, a variable number of subjects, mark validation from 0 to 100 inclusive, total and average calculations, letter-grade assignment, result display, and an option to process another student.

Excluded: database storage, user accounts, editing saved records, web interface, report export, and multiple grading policies.

1.2 Assumptions

Each subject has one numeric mark. All subjects have equal weight in the average. At least one subject is required. The grading scale is proposed for this assignment and is not claimed to be an institutional policy.

2. Requirements Analysis (Job 1)

Requirements analysis identifies what the system must do and the constraints it must satisfy before implementation.

ID	Requirement	Category
FR1	Accept a non-empty student name.	Functional
FR2	Ask the user for the number of subjects.	Functional
FR3	Accept a subject name and mark for each subject.	Functional
FR4	Accept only numeric marks from 0 through 100 inclusive.	Functional
FR5	Reject empty names, non-numeric marks, and invalid subject counts.	Functional
FR6	Calculate the sum of all subject marks.	Functional
FR7	Calculate the arithmetic mean of the marks.	Functional
FR8	Assign a letter grade using the proposed grading scale.	Functional
FR9	Display the student, subjects, marks, total, average, and grade.	Functional
FR10	Offer the user the option to process another student.	Functional
NFR1	Show clear validation messages and recover from invalid entries without terminating.	Non-functional
NFR2	Provide a simple text-based interaction.	Non-functional

2.1 Proposed grading scale

Average mark	Letter grade
90 - 100	A
80 - 89.99	B
70 - 79.99	C
60 - 69.99	D
50 - 59.99	E
Below 50	F

The unrounded average determines the grade. The displayed average is rounded to two decimal places.

3. Design (Job 2)

The design separates the program into small responsibilities so that each part can be understood and tested independently.

Function / component	Responsibility
get_non_empty_text	Read text and repeat until it is not empty.
get_positive_integer	Read the subject count and require a whole number greater than zero.
collect_subjects	Collect the user-selected number of subject names and marks.
get_valid_mark	Require a numeric mark between 0 and 100 inclusive.
calculate_results	Calculate total, average, and grade.
get_grade	Map an average to the proposed letter grade.
display_results	Print the student's subjects and calculated results.

3.1 Data structure

Subjects are stored in a list of two-element tuples. Each tuple contains a subject name and its mark, for example {"Mathematics", 85.0}. A list supports a variable number of subjects, while each tuple keeps a subject paired with its mark.

```
[{"Mathematics", 85.0}, {"Programming", 92.0}, {"English", 76.0}]
```

3.2 Process sequence

Step	Process
1	Request the student's name.
2	Request a positive whole number for the subject count.
3	For each subject, request its name and mark.
4	If an entry is invalid, display an error and request it again.
5	Calculate total and average after all subjects are collected.
6	Determine the letter grade using the average.
7	Display the result summary.
8	Ask whether to process another student; repeat or exit.

4. Implementation (Job 3)

The program uses Elixir functions, pattern matching, guards, recursion, lists, tuples, and Enum functions. The complete source code is supplied separately as student_grade_management_system.exs.

4.1 Implementation notes

- Recursive input functions repeat prompts until valid data is provided.
- collect_subjects/3 gathers the number of subjects selected by the user.
- A list of tuples stores each subject with its mark.
- Enum.reduce/3 calculates the total, and length/1 determines the number of subjects.
- Function clauses with guards assign grades from the highest threshold downward.

4.2 How to run

1. Install Elixir and Erlang/OTP.
2. Save the source as student_grade_management_system.exs.
3. Open a terminal in the file's directory.

4. Run: `elixir student_grade_management_system.exs`

5. Testing (Job 4)

Each test case was run against the program and the observed result was recorded.

Test	Input / condition	Expected result	Result
T1	Marks 80, 90, 70	Total 240; average 80.00; grade B.	Passed
T2	One subject with mark 100	Total 100; average 100.00; grade A.	Passed
T3	Average exactly 90	Grade A.	Passed
T4	Average exactly 80	Grade B.	Passed
T5	Average exactly 50	Grade E.	Passed
T6	Mark -1 or 101	Reject and prompt again.	Passed
T7	Non-numeric mark, e.g. abc	Reject and prompt again.	Passed
T8	Subject count zero, negative, or non-integer	Reject and prompt again.	Passed
T9	Empty student or subject name	Reject and prompt again.	Passed
T10	Choose to process another student	Restart input sequence.	Passed

6. Documentation (Job 5)

Run the program using the Elixir command in Section 4.2. Enter a student name, a positive subject count, then each subject name and a mark from 0 to 100. The program displays the marks, total, average to two decimal places, and letter grade. Enter y or yes to process another student; any other response exits.

7. Software Process Measurement

An estimate is a prediction made before starting a job. Actual time is recorded after performing it. A work-product unit is a countable output of a job, such as a requirement, design artifact, function, or test case. Units are defined separately for each job and are not added across unlike categories.

7.1 Job Number Log

Job	Work-product unit	Estimated time	Actual time	Estimated units
1. Requirements	Requirements documented	45 min	30 min	10 requirements
2. Design	Design artifacts: flowchart and function plan	30 min	25 min	2 artifacts
3. Implementation	Program functions implemented	60 min	47 min	11 functions
4. Testing	Test cases executed	35 min	31 min	10 test cases
5. Documentation	Documentation sections prepared	30 min	20 min	4 sections
Total time		200 min	153 min	Units are not additive

7.2 Daily Time Log: 4 November 2026

Date	Job	Activity	Est. min	Actual min
04-Nov-2026	1	Identify scope and document requirements	45	30
04-Nov-2026	2	Choose data structure and define program flow/functions	30	25
04-Nov-2026	3	Implement input, validation, calculations, and output	60	47
04-Nov-2026	4	Run test cases and correct defects	35	31
04-Nov-2026	5	Prepare user and technical documentation	30	20
		Total	200	153

8. Limitations and Possible Improvements

This version keeps data only while the program is running. It does not save records to a file or database. It assumes all subjects have equal weight and uses one proposed grading scale. Future improvements could include saving records, editing marks, generating class reports, supporting weighted subjects, and allowing configurable grading policies.

9. Conclusion

The project defines and implements a small command-line Student Grade Management System in Elixir. Its design uses functions and a list of subject-mark tuples to support a variable number of subjects. Input validation protects calculations from invalid marks and empty names. The project also applies process measurement by identifying jobs and estimating time and work-product units for each.